

Turning the DhakaSim work into an MSc thesis in Data Science

Written against the six papers in [DhakaSIM Papers/](#) and the state of the Python port in this repository. Compute figures come from sweeps actually run on this machine, not from estimates.

Where you are starting from

Your earlier work looked at flow diversion on a single route through one four-legged intersection, using lane diversion and intersection blocking. As a motivating case that is fine. As a thesis it has two problems. A single intersection yields a handful of scenarios, which is not enough evidence to support a general claim, and there is no model in it, so a Data Science examiner will ask where the learning is.

The fix is to keep the question and change both the scale and the method. Diversion stays the subject. What changes is that it happens across a network rather than one junction, and that a model decides where to divert instead of you enumerating cases by hand. That also gives your thesis a clean narrative: you showed the effect on one intersection, and the thesis generalises it.

What the existing work leaves open

Across the six papers, nothing learns. The 2024 RoadBird paper compares fixed configurations. The 2025 side friction paper measures the impact of static elements. The 2025 signal scheduling paper optimises, but with NSGA-II, which searches a space rather than building a model of it, and what it optimises is signal timing, not diversion.

Paper	What it optimises or learns	What it leaves for you
2024, IEEE T-ITS, lane vs non-lane	Nothing. Compares fixed configurations	No policy, no learned model, one global lane setting for the whole network
2025, IEEE Access, side friction	Nothing. Measures static impact	Side friction is never used as a control variable
2025, IEEE Access, signal scheduling	NSGA-II over signal timings, offline	Search, not a model. Signals only, no diversion or blocking
2020, SIMPAT, GIS parsing	Nothing. Geometric algorithm	Networks stay small, never city scale

So three things are open at once: no learned model of the simulator, no diversion policy of any kind, and no network scale treatment of either. Those three gaps line up with each other, which is what makes a coherent thesis rather than three unrelated chapters.

The proposal

A working title: *Learning to divert: a graph neural surrogate and policy search for traffic diversion in non-lane-based heterogeneous road networks.*

Three contributions, in dependency order.

1. Diversion and blocking as network-scale controls

At the moment strip width is a single global setting, so a network is either entirely lane-based or entirely not. The first piece of work is to make diversion, blocking and lane discipline properties of individual links and intersections rather than of the whole run. This is a modest change to the simulator and it is what turns your single-intersection study into something that can be posed as a decision over a network.

This chapter is engineering rather than science, and it should be written that way. Its value is that it creates the action space the rest of the thesis searches over.

2. A graph neural surrogate of the simulator

Train a model that takes a network, a demand pattern and a diversion configuration, and predicts per-link speed, waiting time and flow without running the simulator.

This is the Data Science core, and it is the part that solves a practical problem rather than merely satisfying a degree requirement. The simulator is slow, and any search over diversion configurations needs thousands of evaluations. A surrogate that answers in milliseconds is what makes the third contribution possible at all.

The road network is already a graph, so the representation is natural. Links become nodes, junctions become edges between them, and link attributes carry width, length, strip count, demand and the current diversion state. A message-passing architecture such as GraphSAGE or a graph attention network is the obvious starting point, because congestion propagates between adjacent links and that is exactly what message passing captures.

3. Policy search over diversion configurations

With a fast surrogate you can search. Two routes are open, and I would suggest deciding between them only after the surrogate exists.

The cheaper route is surrogate-guided evolutionary search, which gives you a direct comparison against the 2025 NSGA-II paper using the same family of algorithm applied to a different control variable. The more ambitious route is reinforcement learning, where an agent adjusts diversion in response to the evolving state of the network. Reinforcement learning is a crowded field for traffic signal control, so the novelty has to rest on the non-lane heterogeneous setting and on diversion as the action, not on the algorithm.

Whichever you choose, every configuration the search proposes as good must be re-run in the actual simulator before you report it. The surrogate is a search accelerator, not evidence.

The compute budget, measured

This is the part most simulation theses get wrong, so plan it early. The figures below come from sweeps run on this machine, an 8 core processor with 8 jobs in parallel.

Configuration	Median run	Slowest run	Note
66/100/120 veh/h per OD pair, 1800 s	388 s	1175 s	The simulator's own demand levels
400 veh/h per OD pair, 1800 s	about 2800 s		Roughly three times the built-in high setting
Full 72 run sweep	3889 s wall	30339 s CPU	Four factors, three seeds

A full 240 run grid at the higher demand levels takes about a day. Scale that to the tens of thousands of runs a deep model wants and you are looking at months of continuous compute, which you do not have.

The single most useful thing you can do early

Instrument the simulator to log per-link state at intervals rather than only writing an average at the end of the run. A 1800 second run sampled every 10 seconds turns one training example into 180 time steps across every link. On the 23 link Dhaka network that is roughly four thousand link-observations from a single run instead of 23. The same compute then yields two orders of magnitude more supervision, and you get temporal structure, which lets the surrogate predict how congestion builds rather than only where it ends up.

Three further economies are worth knowing. Shorter runs are disproportionately cheap, because cost grows faster than linearly with duration as the network fills. Smaller synthetic networks train the surrogate on topological variety without the cost of a large one. And the harness in this repository already resumes interrupted sweeps and runs them in parallel, so generation can proceed in the background over weeks.

Evaluation, and the mistakes to avoid

Two failures would sink the thesis at viva, and both are easy to avoid if you design for them now.

The first is data leakage. If you pool every link observation and split randomly, observations from the same run land on both sides of the split and your reported accuracy is meaningless. Split by scenario, and better still hold out an entire topology and an entire demand level, so that you are measuring whether the surrogate generalises rather than whether it memorises.

The second is confusing the simulator with reality. Your surrogate learns the simulator. The simulator itself was validated against observed travel times on two Dhaka routes at about 88 percent accuracy by MAPE. Any claim about real roads inherits that error and should be stated with it. Say plainly that you are optimising against a validated model, not against Dhaka.

For baselines, include a no-diversion control, a random diversion control, ridge regression and gradient boosting on hand-built features, and, if you can afford the runs, NSGA-II applied directly to the simulator. That last one is the comparison a reviewer from the 2025 paper's group will want to see, because it answers whether the surrogate bought you anything.

The error measures and hypothesis tests you need are already implemented in `experiments/stats.py`, with no external dependency: mean error, mean absolute error, root mean square error, MAPE, RMSPE, a two sample t-test and a two sample Kolmogorov-Smirnov test. Those are the same measures the 2024 paper reports in its Table IV, which makes your numbers directly comparable to published ones.

Where the data mining fits

If your programme wants data mining specifically rather than only supervised learning, the generated sweep data supports it without extra simulation.

Cluster links by how they respond across configurations, and you get a typology of road behaviour: links that saturate early, links that only matter under diversion, links that never bind. Attribution methods over the trained surrogate, whether permutation importance or a graph-specific explainer, then tell you which links dominate network performance. That is a genuinely useful planning output, because it identifies the small set of interventions worth paying for, and it connects back to your original question about which route to divert.

Risks, and what to do about them

Risk	Mitigation
Reinforcement learning needs more episodes than you can simulate	Train the agent against the surrogate, not the simulator. Fall back to evolutionary search, which needs far fewer evaluations
The surrogate does not generalise to unseen topologies	Train across several: the Dhaka network, the Miami and Riyadh grids, and synthetic grids from <code>make_grid.py</code>
Changes to the simulator break its arithmetic parity with the Java original	Run <code>tests/test_javacompat.py</code> after every change. It exists for this
Scope grows past one degree	Contributions one and two alone are a complete thesis. Treat the policy search as the part you cut if time runs short

A smaller version, if the full plan is too much

Drop the third contribution. Build the diversion controls, generate the data, train the surrogate, and use it for sensitivity analysis rather than optimisation: which links, under which demand, most affect network performance. That is still novel against all six papers, still unambiguously a Data Science thesis, and it needs no reinforcement learning at all. You can then present the policy search as future work, or as a short final chapter if the calendar allows.

Suggested order of work

1. Reproduce a baseline sweep on the Dhaka network and confirm you can regenerate the existing comparison report. This costs about an hour and proves the pipeline works end to end.
2. Add per-link state logging at intervals. Do this before generating any training data, because regenerating it later is expensive.
3. Make diversion, blocking and strip width per-link properties. Write the tests alongside.

4. Generate training data continuously in the background from here on. It is the long pole and it parallelises with everything else.
5. Build the surrogate. Start with a plain graph network on end-of-run averages, then add the temporal component.
6. Evaluate the surrogate properly, with held-out topologies and demand levels.
7. Only then attempt policy search, and validate its output in the simulator.

Publication targets

A thesis in this shape splits naturally into two papers, which is how this research group publishes and is easier than placing one paper that does everything.

The surrogate is a methods contribution and fits Simulation Modelling Practice and Theory, which published the group's GIS work, or IEEE Access, which took both 2025 papers and is fast and open access. The diversion findings are an applied contribution and belong in IEEE Transactions on Intelligent Transportation Systems or Transportation Research Part C, the latter being particularly receptive to machine learning in transport. Expect a long cycle at T-ITS: the 2024 RoadBird paper was submitted in November 2022 and published in June 2024.

For conferences, IEEE ITSC is the main annual venue in the field. The Winter Simulation Conference suits the surrogate and data generation methodology. Locally, ICCIT and NSysS are worth using to get feedback on the approach before committing to a journal submission.

What to settle before writing a proposal

Three decisions shape everything else, and none of them needs more reading to answer.

Decide whether your networks come from real GIS extracts or synthetic grids. Real extracts make the applied claim stronger and cost you time in conversion, where the published parser reached 86 percent accuracy. Synthetic grids give you topological variety cheaply, which the surrogate needs. Most likely you want both, real for the headline result and synthetic for training.

Decide whether you need field data. If you want to claim anything about real Dhaka roads beyond what the simulator already establishes, you need observed travel times, and collecting them is a project in itself. If you are content to optimise against a validated simulator, you do not, and I would suggest you are not, because the thesis is strong without it.

Decide how much of the four-legged intersection work carries forward. Using it as a validation case, where the learned policy should recover the diversion you already found by hand, would be a neat result and a good early milestone. It also gives your examiners continuity between your previous work and this one.

Prepared from the seven PDFs in DhakaSIM Papers and measured runs of the Python port in this repository. Compute figures are from 72 completed simulations on an 8 core machine. Accuracy figures attributed to published papers are theirs, not measurements made here.